MyCodeSchool

L2 · Metode HTTP cu Postman

Câte o colecție pentru fiecare aplicație: te joci cu cererile, apoi le pui teste

Azi nu construiești nimic nou. Pornești din VS Code cele cinci aplicații și vorbești cu ele din **Postman**. Lucrezi în două etape: **întâi te joci** (schimbi o cerere, o trimiți, vezi ce răspunde serverul și notezi), **apoi scrii teste** care verifică singure ce ai descoperit. Un DevOps face asta zilnic: verifică un deploy, găsește de ce pică o aplicație, dovedește că „merge” cu o rulare, nu cu o părere.

Fiecare parte are un criteriu de gata. Răspunsurile experimentelor nu sunt în fișă: le descoperi tu și le scrii în `notite-L2.txt`, în rădăcina repo-ului, câte un rând, cu cuvintele tale. De exemplu: `one-model 1: 200, vine mașina mea`.

Ce trebuie să știi

- **Cerere și răspuns**: clientul (Postman, browserul) trimite o cerere, serverul (API-ul) întoarce un răspuns. Mereu în perechi.
- **URL-ul**: `http://localhost:5017/api/cars/2?color=alb` = `host` : `port` / `cale` (cu `2` = id-ul) ? `query` (filtre `cheie=valoare`, unite cu `&`). Un parametru de query pe care endpoint-ul nu-l cunoaște este ignorat.
- **Corpul** (JSON) e datele trimise la `POST`, `PUT`, `PATCH`. Header-ul `Content-Type: application/json` spune serverului că ce trimiți e JSON.
- **Idempotent** = o repeți de 10 ori și rezultatul e același ca după prima dată. `GET`, `PUT`, `DELETE` sunt; `POST` nu e.

Metoda	Ce face	Corp?
`GET`	citește	nu
`POST`	creează o resursă nouă	da
`PUT`	înlocuiește resursa întreagă	da
`PATCH`	schimbă doar câmpurile trimise	da
`DELETE`	șterge	nu

Cod	Înseamnă
`200` / `201` / `204`	a mers / s-a creat (header-ul `Location` spune unde) / a mers, fără corp
`400`	cererea ta e greșită
`401` / `403`	serverul nu știe cine ești / știe, dar nu ai voie
`404` / `405`	resursa nu există / metoda nu există pe adresa aceea
`409` / `415`	ar ieși un duplicat / tipul corpului nu e acceptat
`5xx`	a greșit serverul, nu tu (`503` = nu poate lucra acum)

Cuvintele din Postman

Cuvânt	Ce e
Workspace	spațiul tău de lucru; azi unul, `devops-initiere`, cu tot ce urmează
Colecție	cererile salvate ale unei aplicații; se împarte în foldere
Environment	variabile pentru un mediu (aici: adresa aplicației, `baseUrl`). Mâine aceleași cereri merg pe un server din AWS: schimbi doar mediul
Variabilă de colecție	o valoare ținută minte între cereri (ex. id-ul mașinii create). O scrie un script
Scripts → Post-response	cod JavaScript care rulează *după* ce vine răspunsul; aici scrii testele
Collection Runner	rulează toată colecția, cerere după cerere, și arată ce teste au trecut

Pregătire — pornești cele cinci aplicații din VS Code

Aplicațiile sunt în `proiecte/`. Fiecare are (unde trebuie) un fișier compose doar cu MySQL, pe un port al lui, și un port al aplicației în `Properties/launchSettings.json`:

Aplicația (folder)	Fișier compose cu MySQL	MySQL	Aplicația
`car-source-api-csharp`	nu are nevoie de bază	–	`5643`
`one-model-api-csharp`	`docker-compose-localOne.yaml`	`3500`	`5017`
`cars-users-api-csharp`	`docker-compose-local.yaml`	`3310`	`5648`
`online-shop-api-csharp`	nu există încă (vezi mai jos)	`3340`	`5082`
`online-school-api-csharp`	`docker-compose-school-api-csharp.yaml`	`3330`	`5612`

Pentru fiecare aplicație, în ordine:

1. În terminalul din VS Code, în folderul proiectului, pornești MySQL (dacă are fișier compose): `docker compose -f NUME-FIȘIER up -d`. Așteaptă ~30 de secunde, până `docker compose -f NUME-FIȘIER ps` arată `(healthy)`.
2. În `appsettings.Development.json`, la `DefaultConnection`, pui în `port=` portul MySQL din tabel (la cars-users e deja pus).
3. Într-un terminal nou, în folderul proiectului: `dotnet run`. Rămâne pornit cât lucrezi; la prima pornire aplicația își creează singură tabelele (la shop și school și datele de test).
4. În browser: `http://localhost:PORT/swagger`.

Pentru `online-shop` copiezi în folderul lui fișierul `docker-compose-school-api-csharp.yaml`, îl redenumești `docker-compose-shop-api-csharp.yaml` și schimbi portul `3330:3306` în `3340:3306`. Ai deci cinci terminale cu `dotnet run`, iar Swagger-ul fiecăreia e lista ei de endpoint-uri: o vei folosi în exerciții.

> **ATENȚIE**
>
> Proiectele sunt `net8.0`. Dacă `dotnet run` spune că lipsește runtime-ul 8.0, instalezi **.NET 8 SDK**; nu schimbi `TargetFramework` din proiect. Docker Desktop trebuie să fie pornit pentru MySQL. Dacă aplicația scrie că nu se poate conecta la MySQL, portul din `appsettings.Development.json` nu e cel din compose.

> **CRITERIU DE GATA**
>
> Swagger se deschide în browser pe toate cele cinci porturi (5643, 5017, 5648, 5082, 5612).

Instalezi Postman și îți faci workspace-ul

1. Descarcă și instalează Postman pentru Windows de pe `postman.com` (aplicația desktop). La prima pornire îți face cont (gratuit) sau te lasă să intri cu Google ori GitHub.
2. Creează un workspace nou, gol (*blank*), numit `devops-initiere`, din selectorul de workspace-uri din partea de sus a ferestrei. Toate colecțiile de azi stau în el.

Partea 1 — One-model, ghidat

Facem împreună colecția pentru `one-model` (mașini, port `5017`). Numește cererile ca până acum: `getAllCars`, `addCar` și tot așa.

1. Colecția și environment-ul

1. În workspace creează colecția `one-model`.
2. Creează un environment `one-model local` cu o variabilă `baseUrl` = `http://localhost:5017` (completează și valoarea inițială, și pe cea curentă) și salvează-l.
3. Alege-l ca environment activ, din selectorul din colțul dreapta-sus. Fără el, `{{baseUrl}}` rămâne necompletat.

2. Primele cereri

Adaugă `getAllCars`: metoda `GET`, adresa `{{baseUrl}}/api/cars`, apoi *Send*. Sub cerere vezi codul, corpul și, într-un tab separat, header-ele răspunsului. Apoi `addCar`: `POST` pe aceeași adresă; la tab-ul *Body* alegi *raw*, iar în dreapta tipul **JSON** (Postman adaugă singur `Content-Type: application/json`):

```
{
  "brand": "Dacia", "model": "Logan", "color": "alb", "size": 5,
  "year": 2020, "price": 8000, "mileage": 90000,
  "fuelType": "benzina", "transmission": "manuala",
  "available": true
}
```

Trimite-o **o singură dată**. Uită-te la cod, la corp și la header-ul `Location`. Mai adaugă, cu adrese din Swagger: `getCarById` (cu id-ul primit), `getCarBySearch`, `putCar`, `patchCar`, `deleteCar`.

3. Joaca

Pentru fiecare rând: schimbi cererea cum scrie, o trimiți, te uiți la cod și la mesaj și notezi în `notite-L2.txt` (`one-model 7: …`). Unde scrie „mașina ta”, folosești id-ul mașinii create mai sus. Întrebarea de fiecare dată e aceeași: *de ce a răspuns așa?*

Nr	Schimbi, trimiți, notezi codul și mesajul
1	GET pe id-ul unei mașini create de tine; uită-te la tab-ul Headers al răspunsului
2	GET cu un id care nu există: 99999
3	GET cu un id care nu e număr: abc
4	GET /api/cars/search cu un parametru inventat: ?ceva=test
5	GET /api/cars/search?minYear=2025&maxYear=2019
6	DELETE pe /api/cars, fără id
7	POST cu Body-ul gol: {}
8	POST cu JSON stricat: lipsește acolada de la final
9	POST fără câmpul model
10	POST cu year scris ca text: "douamii"
11	POST cu year: 1800
12	POST cu price negativ
13	POST cu Body de tip Text, nu JSON
14	POST cu exact aceeași mașină (Dacia Logan) a doua oară
15	PUT pe mașina ta, cu un singur câmp: {"price": 8500}
16	PATCH pe mașina ta, cu un singur câmp: {"price": 8500}
17	PATCH cu {}
18	PATCH cu price scris ca text: "mult"
19	DELETE pe mașina ta
20	DELETE pe aceeași mașină, a doua oară

4. Testele și variabila pentru id

Acum fac cererile singure verificarea. Întâi, ca să nu mai scrii id-ul de mână, în cererea `addCar` deschide tab-ul *Scripts* și alege **Post-response** (rulează după ce vine răspunsul):

```
pm.test("201 Created", () => {
  pm.response.to.have.status(201);
});
const car = pm.response.json();
pm.collectionVariables.set("carId", car.id);
```

`pm.test("nume", () => {…})` e un test; `pm.response.json()` e corpul răspunsului ca obiect; ultima linie ține minte id-ul în variabila de colecție `carId`. În celelalte cereri scrii acum `{{carId}}` în loc de id. Mașina din joacă a fost ștearsă la rândurile 19–20, deci trimite `addCar` din nou. După *Send*, la rezultatele testelor apare `PASS 201 Created`, iar variabila `carId` apare în tab-ul *Variables* al colecției.

Același tipar la `getCarById` (cod și un câmp din corp) și la filtre/căutare (o listă):

```
pm.test("200 si e masina cautata", () => {
  pm.response.to.have.status(200);
  pm.expect(pm.response.json().brand).to.eql("Dacia");
});

const cars = pm.response.json().cars;
pm.test("200 si toate masinile sunt Dacia", () => {
  pm.response.to.have.status(200);
  pm.expect(cars).to.not.be.empty;
  pm.expect(cars.every(c => c.brand === "Dacia")).to.be.true;
});
```

Pentru rândurile de eroare din joacă, testul are aceeași formă, cu codul pe care l-ai văzut tu. Transformă-le în cereri salvate, într-un folder `cereri greșite`, fiecare cu testul ei. La cele cu `errors`, verifici și ce câmp e numit (în loc de `NumeCamp` pui numele văzut în `errors`):

```
pm.test("400 Bad Request", () => {
  pm.response.to.have.status(400);
  pm.expect(pm.response.json().errors).to.have.property("NumeCamp");
});
```

Ordinea în bară = ordinea din Runner. Pune cererile așa: `addCar`, citirile, `putCar`, `patchCar`, cererile greșite, apoi la final `deleteCar`, un `getCarById` care acum trebuie să dea `404` și încă un `deleteCar`. Ștergerea de la final îți **curăță datele**: fără ea, a doua rulare ar da `409` la `addCar`. Adaugă și un folder `health`, cu `GET {{baseUrl}}/health/live` și `/health/ready`, fiecare cu testul `200` (pe adresele astea întreabă un orchestrator dacă aplicația trăiește).

5. Collection Runner

Pe colecție, rulezi Runner-ul (butonul *Run* al colecției, sau meniul ei). Alege environment-ul `one-model local` și pornește rularea. Runner-ul trimite cererile în ordinea din bară și arată, sub fiecare, testele trecute sau picate. Rulează-l de două ori la rând.

> **CRITERIU DE GATA**
>
> Rândurile 1–20 din tabel sunt în `notite-L2.txt`. Runner-ul one-model e verde de două ori la rând. Poți explica, pe datele tale: de ce a doua rulare nu dă `409`; de ce `PUT` cu un singur câmp și `PATCH` cu un singur câmp răspund diferit; ce arată `Location`.

> **DE ȚINUT MINTE**
>
> Pentru erorile aruncate de aplicație (cele cu `detail`), log-ul din terminalul `dotnet run` scrie `StatusCode: 200`, deși clientul a primit alt cod. Vezi singură, în terminalul aplicației, după un `404`. Starea reală o arată răspunsul din Postman și testele tale, nu doar log-ul.

Când nu merge

Vezi	De ce	Ce faci
Postman: *Could not get response / ECONNREFUSED*	Aplicația nu rulează sau portul e greșit	Uită-te în terminalul `dotnet run`; deschide `/swagger` în browser
Adresa rămâne `{{baseUrl}}/…` sau iese *Invalid URL*	Nu e ales environment-ul	Selectorul din colțul dreapta-sus
`dotnet run` scrie *Unable to connect to any of the specified MySQL hosts*	MySQL nu e pornit sau portul din `appsettings.Development.json` nu e al compose-ului	`docker compose -f FIȘIER ps`; compari portul
`dotnet run` scrie *address already in use*	Aceeași aplicație rulează deja într-alt terminal	O oprești cu `Ctrl+C` acolo
`409` la `POST` în Runner	Ce ai creat la joacă a rămas în bază	Ștergi resursa cu `DELETE` și rulezi iar

`415` la un `POST` care ar trebui să meargă	Corpul nu e trimis ca JSON	Body → raw → tipul **JSON**
Testul nu apare în rezultate	Scriptul e în Pre-request, nu în Post-response; sau nu ai salvat	Scripts → Post-response; `Ctrl+S`
`401` pe tot, după un timp, la o aplicație cu token	Token-ul a expirat	Rulezi din nou cererea de login

Partea 2 — Exerciții, singură

La fiecare aplicație faci la fel ca la one-model: environment cu `baseUrl`, colecție cu foldere, apoi **joaca** (tabelul: schimbi, trimiți, notezi în `notite-L2.txt`) și apoi **testele**: o cerere salvată pentru fiecare endpoint din Swagger, un test la fiecare, cererile greșite din joacă păstrate ca teste, date lăsate curate la final (ce creezi, ștergi). Fiecare aplicație are și `/health/live` și `/health/ready` (nu apar în Swagger): le pui într-un folder `health`, cu testul `200`.

Exercițiul 1 — car-source (`5643`)

O sursă externă de mașini, fără bază de date: lista se generează la pornire și se poate regenera. Swagger arată 3 endpoint-uri. Id-ul unei mașini (un text) îl iei din lista primită, cu un script, nu îl scrii de mână.

Nr	Schimbi, trimiți, notezi codul și mesajul
1	GET /api/external/cars și uită-te la lungimea listei
2	GET cu un externalId care nu există: nu-exista
3	GET /api/external/cars?count=5 (parametru care nu aparține acestui endpoint)
4	POST /api/external/cars/regenerate, fără count
5	POST regenerate cu count=0
6	POST regenerate cu count=1001
7	POST regenerate cu count=abc
8	POST regenerate cu parametrul scris cu literă mare: Count=3
9	POST regenerate cu parametrul scris greșit: cont=3
10	GET pe adresa /regenerate (metoda greșită)
11	DELETE pe /api/external/cars
12	POST pe /api/external/cars, cu un Body oarecare
13	Rămâne lista cum era: POST regenerate cu count=120

> **CRITERIU DE GATA**
>
> Rândurile 1–13 sunt în `notite-L2.txt`. Runner-ul car-source e verde de două ori la rând, cu o cerere pentru fiecare endpoint din Swagger, folderul `health` și cele mai multe cereri greșite din tabel ca teste. Un test verifică numărul de mașini după o regenerare, iar colecția lasă serverul cu lista cum era.

Exercițiul 2 — cars-users (`5648`)

Colecția `cars-users-management-api` pe care ai început-o o **completezi**, nu o refaci. Utilizatori și mașinile lor (un utilizator are mai multe mașini). Swagger arată 18 endpoint-uri.

1. Creează environment-ul `cars-users local` cu `baseUrl` = `http://localhost:5648` și înlocuiește în cererile tale începutul adresei scris de mână cu `{{baseUrl}}`.
2. Adaugă cererile lipsă, cu numele în stilul tău (`getUserById`, `putCar` …), până ai câte una pentru fiecare endpoint.
3. Variabilele merg și în corpul unei cereri: `"userId": {{userId}}`, fără ghilimele în jurul variabilei. Creezi un utilizator, salvezi `userId`, apoi o mașină care îi aparține.
4. Joaca:

Nr	Schimbi, trimiți, notezi codul și mesajul
1	GET /api/users/99999
2	GET /api/users/abc
3	GET /api/users/99999/cars (utilizator care nu există)
4	GET /api/cars?ceva=test (parametru inventat)
5	GET /api/cars/search cu parametrul scris greșit: ?Brnad=Dacia
6	GET /api/cars/search?UserId=99999

7	GET /api/cars/by-year-range, fără niciun parametru
8	GET /api/cars/by-year-range?minYear=abc&maxYear=2020
9	GET /api/cars/by-year-range?minYear=2025&maxYear=2019
10	GET /api/users/by-email/nimeni@example.com
11	POST /api/users cu email: "nu-e-email"
12	POST /api/users cu name: "" (șir gol)
13	POST /api/users cu emailul unui utilizator care există deja
14	POST /api/cars cu userId care nu există: 99999
15	POST /api/cars cu userId: 0
16	POST /api/cars cu userId scris ca text: "ana"
17	POST /api/cars cu Body de tip Text, nu JSON
18	PUT /api/cars/{id} fără câmpul year
19	PATCH /api/users/{id} cu {}
20	PATCH /api/users/{id} cu emailul altui utilizator: bob.exp@example.com
21	PATCH /api/cars/{id} cu userId-ul celuilalt utilizator
22	DELETE pe mașina ta; compară răspunsul cu DELETE de la one-model
23	DELETE pe aceeași mașină, a doua oară

Ordinea pentru Runner: utilizator, mașină, citiri, modificări, cereri greșite, apoi ștergi mașina și utilizatorul. Baza are deja date de dinainte (utilizatori și mașini); testele tale verifică *ce ai creat tu* (după id), nu numărul total.

> **CRITERIU DE GATA**
>
> Rândurile 1–23 sunt în `notite-L2.txt`. Runner-ul cars-users e verde de două ori la rând, cu toate endpoint-urile din Swagger. Testul de la fiecare `DELETE` verifică *corpul* răspunsului. Cu MySQL-ul acestei aplicații oprit (`docker compose -f docker-compose-local.yaml stop`) rulezi doar folderul `health` (Runner-ul poate rula și un singur folder) și poți spune care dintre `live` și `ready` pică și de ce. Apoi pornești MySQL la loc (`start`).

Exercițiul 3 — online-shop (`5082`)

Primul serviciu cu autentificare. Aproape toate endpoint-urile cer un **token** (JWT), primit la login și trimis în fiecare cerere. În plus, fiecare endpoint cere o **permisiune**, pe care token-ul o conține sau nu. Conturi de test (le creează seed-ul):

Cont	Parola	Permisiuni
`admin@shop.com`	`admin1234`	toate (`USER_*`, `PRODUCT_*`, `ORDER_*`, `PERMISSION_*` …)
`customer@shop.com`	`customer1234`	de client: `PRODUCT_READ`, `ORDER_READ`, `ORDER_WRITE`, `ORDER_DETAILS_READ`, `ORDER_DETAILS_WRITE`

Pasul nou: token-ul. Cerere `POST /api/auth/login`, Body JSON cu `email` și `password`. Pentru joacă, copiezi `accessToken` din răspuns și îl lipești în tab-ul *Authorization* al cererii următoare: tipul *Bearer Token*. Pentru teste, îl salvezi automat, în *Post-response* de la login:

```
const auth = pm.response.json();
pm.collectionVariables.set("token", auth.accessToken);
```

Apoi, pe **colecție**, tab-ul *Authorization*: tipul *Bearer Token*, valoarea `{{token}}`; cererile îl moștenesc (*Inherit auth from parent*). Cererea de login are *No Auth*. Loginul clientului îl salvezi în `tokenCustomer`, iar cererile marcate `[client]` au, în propriul tab *Authorization*, Bearer cu `{{tokenCustomer}}`. Environment-ul `online-shop local` ține `baseUrl` și conturile (`adminEmail` …). Pentru un email care nu există încă, îl compui în *Scripts* → *Pre-request* (rulează *înainte* de cerere):

```
pm.collectionVariables.set("regEmail",
  "ana." + Date.now() + "@example.com");
```

Swagger arată 41 de endpoint-uri în 8 grupuri (auth, admins, customers, products, orders, order-details, permissions, users); câte un folder pe grup. Id-urile sunt GUID-uri (text lung). Cererile de login sunt primele din colecție. Joaca (unde nu scrie `[client]`, ești adminul):

Nr	Schimbi, trimiți, notezi codul și mesajul
1	GET /api/products fără niciun token (Authorization: No Auth)

2	GET /api/products cu token inventat: x.y.z
3	GET /api/products cu refreshToken în loc de accessToken
4	POST /api/auth/login cu parola greșită
5	POST /api/auth/login fără câmpul password
6	POST /api/auth/login cu email care nu arată a email
7	POST /api/auth/refresh cu un refreshToken inventat
8	POST /api/auth/register cu emailul unui cont existent: customer@shop.com
9	POST /api/auth/register cu parola de 3 caractere
10	[client] GET /api/users
11	[client] GET /api/products (același client, alt endpoint)
12	[client] POST /api/products
13	[client] GET /api/customers/{id-ul altui client}
14	[client] GET /api/orders/{id-ul unei comenzi a altui client}
15	[client] GET /api/admins/me
16	GET /api/products/{GUID inexistent}
17	GET /api/products/abc
18	GET /api/products?size=abc
19	GET /api/products?page=999 (o pagină care nu există)
20	GET /api/products?marime=2 (parametru inventat)
21	POST /api/products cu {}
22	POST /api/products cu price negativ
23	POST /api/products cu stock scris ca text: "mult"
24	POST /api/products valid; uită-te la Location
25	POST /api/orders (admin) fără ?customerId=
26	[client] PATCH /api/orders/{id}/status
27	POST /api/permissions cu name: custom_read (litere mici)
28	POST /api/permissions cu name: PRODUCT_READ (există deja)
29	DELETE /api/products/{id}
30	DELETE /api/products/{id} a doua oară

> **CRITERIU DE GATA**
>
> Rândurile 1–30 sunt în `notite-L2.txt`. Runner-ul online-shop e verde de două ori la rând, cu o cerere pentru fiecare endpoint din Swagger. Ai `401`, `403` și `409` verificate prin teste, o comparație între resursa proprie și cea străină, iar tot ce ai creat e șters la final (atenție la ordinea ștergerilor).

Exercițiul 4 — online-school (`5612`)

Aceeași idee, fără ghidaj la autentificare. O școală online: studenți, profesori, administratori, cursuri, cărți, înscrieri. Parola tuturor conturilor de test e `password`:

Cont	Rol
`admin@online-school.ro`	administrator (toate permisiunile)
`andrei.ionescu@online-school.ro`	profesor
`ana.marinescu@student.online-school.ro`	student

Adresele nu încep cu `/api`; Swagger arată 49 de endpoint-uri în 8 grupuri, cu id-uri numere. Răspunsul de login are alt format decât la magazin: citește-l și găsește tu unde e token-ul. Joaca (`[student]` și `[profesor]` = cu token-ul acelui cont; altfel ești adminul):

Nr	Schimbi, trimiți, notezi codul și mesajul
1	GET /students fără token
2	GET /students cu token inventat: x.y.z
3	POST /auth/login cu parola greșită
4	POST /auth/login fără câmpul password
5	[student] GET /teachers
6	[student] GET /students (același student, alt endpoint)

7	[student] POST /courses
8	[profesor] POST /students
9	[profesor] POST /teachers
10	GET /students/abc
11	GET /students/0
12	GET /students/99999
13	GET /students/find-by-email, fără parametrul email
14	GET /students/search?cautare=Ana (parametrul se numește altfel)
15	POST /students cu emailul unui student care există deja: test.exp@example.com
16	POST /students cu email: nu-e-email
17	POST /students cu firstName de 2 litere
18	POST /students valid; uită-te la Location
19	PUT /students/{id} cu un singur câmp: {"lastName": "Altul"}
20	PATCH /students/{id} cu {}
21	POST /courses cu "department" în loc de "departament"
22	PUT /courses/{id} cu {}
23	GET /courses/find-by-department, fără parametru
24	POST /enrolments cu studentId: 0
25	POST /enrolments cu un student care nu există: 99999
26	POST /enrolments valid; uită-te la Location
27	POST /enrolments cu aceleași valori a doua oară
28	GET /books/find-by-student, fără parametru
29	POST /books cu bookName de 2 litere
30	POST /users/{id}/permissions cu o permisiune care nu există: NU_EXISTA
31	DELETE /users/{id}/permissions/COURSE_WRITE (studentul nu o are)
32	DELETE /users/{id}/permissions/COURSE_WRITE a doua oară
33	DELETE /students/{id}; compară răspunsul cu DELETE de la one-model
34	DELETE /students/{id} a doua oară

Pe lângă tabel: înregistrează un cont nou (`register`), dă-i o permisiune în plus (`/users/{id}/permissions`), loghează-te cu el și arată, printr-un test, că token-ul nou o conține.

> **CRITERIU DE GATA**
>
> Rândurile 1–34 sunt în `notite-L2.txt` . Runner-ul online-school e verde de două ori la rând, cu o cerere pentru fiecare endpoint din Swagger. Ai `401` , `403` și `409` verificate prin teste, un test care arată ce face `PUT` cu un singur câmp la studenți, iar tot ce ai creat e șters la final.

Partea 3 — Livrare: colecțiile intră în repo

O colecție rămasă doar în Postman nu o vede nimeni și nu o poate rula un pipeline. O exporți în repo, ca orice alt cod.

1. În rădăcina repo-ului creezi folderul `postman` .
2. Pentru fiecare colecție: meniul colecției → *Export* → formatul **Collection v2.1** → salvezi în `postman` ca `NUME.postman_collection.json` , cu NUME = `car-source` , `one-model` , `cars-users` , `online-shop` , `online-school` .
3. Pentru fiecare environment: meniul lui → *Export* → salvezi ca `NUME.postman_environment.json` .
4. Din terminalul VS Code:

```
git add postman notite-L2.txt
git commit -m "L2 Postman"
git push
```

> **CRITERIU DE GATA**
>
> Pe GitHub se văd `notite-L2.txt` și, în `postman/` , 10 fișiere: cinci colecții și cinci environment-uri. Verificare: șterge o colecție din Postman, importă-o (*Import*) din fișierul din repo și rulează-o în Runner: tot verde.

ATENȚIE

Environment-urile exportate conțin parolele conturilor de test. Aici e în regulă, sunt date de test pe `localhost`. Într-un proiect real, parolele și token-urile nu intră în Git.

Provocarea — testele rulează fără Postman

Într-un pipeline nu deschide nimeni Postman: colecțiile se rulează din terminal cu **newman**, iar rezultatul e un **exit code** (0 = bine, altceva = pică). Scrie în rădăcina repo-ului un script `ruleaza-teste.sh`, pentru aplicațiile pornite de tine, care:

- verifică mai întâi că fiecare dintre cele cinci aplicații răspunde la `/health/ready` (adresa o ia din environment-ul exportat). Dacă una nu răspunde, scrie care și iese cu cod diferit de 0, fără să ruleze teste;
- rulează cele cinci colecții din `postman/`, fiecare cu environment-ul ei;
- scrie pe ecran doar câte un rând pe colecție, `OK nume` sau `PICA nume`;
- iese cu 0 doar dacă toate colecțiile au trecut.

CRITERIU DE GATA

Cu toate aplicațiile pornite, `./ruleaza-teste.sh; echo $?` scrie cinci rânduri `OK` și `0`. Oprești o aplicație (`Ctrl+C` în terminalul ei): scriptul spune care nu răspunde și iese cu cod diferit de 0. Schimbi într-un fișier exportat un cod așteptat (de ex. `201` în `200`): rândul `PICA` numește colecția, iar codul e diferit de 0. Scriptul e în repo.

ATENȚIE PE WINDOWS

Dacă scrii `.sh` din VS Code pe Windows și bash dă `$'\r': command not found`, fișierul are sfârșituri de rând Windows (CRLF). În VS Code, jos-dreapta, schimbă *CRLF* în *LF* și salvează. Apoi `chmod +x ruleaza-teste.sh`.

CRITERIU DE GATA

Pe GitHub se văd `ruleaza-teste.sh` în rădăcina repo-ului și `notite-L2.txt` cu toate rândurile din cele patru tabele (și al one-model).